

Proiect TSS, anul 3 ID

10 mai 2026

Micu Virgil-Mihai

T1 Testare unitară în Python

- Utilizați un framework de testare unitară din Python pentru a testa funcționalitățile unei clase.
- Ilustrați strategiile de generare de teste prezentate la curs (partiționare în clase de echivalență, analiza valorilor de frontieră, acoperire la nivel de instrucțiune, decizie, condiție, circuite independente, analiză raport creat de generatorul de mutații, teste suplimentare pentru a omorî 2 dintre mutații neechivalenți rămași în viață) pe exemple proprii (create de echipă).

Descrierea clasei testate

Clasa **ContCurent** implementează câteva funcționalități ale unui cont curent

Date membre:

- **Numarul contului**, string din 7 cifre
- **Balanta** (soldul contului): întreg care reprezintă numărul de bani. Pentru a exprima suma în lei se împarte la 100.
- Lista cu **istoricul tranzacțiilor**. Tranzacțiile sunt reprezentate prin clasa **Tranzactie**, care stochează un id de tranzacție, suma (în bani), tipul (1 = încasare, 2 = plată), numărul contului corespondent (din / în care s-a făcut plată)

Metodele clasei:

- **cont_valid** – metoda statică, verifică dacă parametrul furnizat este un număr valid de cont (7 cifre). Este apelată la crearea unei instanțe a clasei și la procesarea tranzacțiilor.
- **balanta** – convertește formatul intern (bani) în lei și returnează rezultatul.
- **procesare_tranzactie** – primește 3 argumente: suma, tipul tranzacției și contul corespondent; procesează tranzacția dacă îndeplinește condițiile necesare:
 1. $\text{round}(\text{suma} * 100)$ este o valoare pozitivă
 2. tipul tranzacției poate fi 1 sau 2
 3. contul corespondent trebuie să fie un număr de cont valid, diferit de variabila număr cont de instanță.
 4. dacă tranzacția este tip plată, valoarea ei să nu depășească balanta contuluiDacă condițiile sunt îndeplinite, balanta contului este actualizată iar tranzacția este adăugată la istoricul contului, cu $\text{id} = \max(\text{id tranzacții din istoric}) + 1$.

Testarea va fi făcută considerând **cont_valid** și **procesare_tranzacții** ca o singură funcție, având în vedere că a doua o apelează pe prima.

Testare functionala

Analiza claselor de echivalenta

Numarul de cont (N) trebuie sa indeplineasca 3 conditii pentru a fi valid

tip = string: da / nu (2 posibilitati)

lungime =7: da / nu (2 posibilitati)

sa contina doar cifre: da / nu (2 posibilitati)

Deci putem distinge 8 clase de echivalenta:

N1 = stringuri de 7 cifre

N2 = stringuri de 7 cifre si / sau alte caractere

N3 = stringuri de $\neq 7$ cifre

N4 = stringuri de $\neq 7$ cifre si / sau alte caractere

N5 = non stringuri de 7 cifre

N6 = non stringuri de 7 cifre si / sau alte caractere

N7 = non stringuri de $\neq 7$ cifre

N8 = non stringuri de $\neq 7$ cifre si / sau alte caractere

Intrucat toate combinatiile sunt posibile si nu stim ce procesari face intern functia consideram ca trebuie testate toate.

Dar stim ca validarea contului se face intr-o functie separata care returneaza true/false, deci pentru testarea functiei de procesare a tranzactiilor vom considera clase de echivalenta 'externe':

NE1 = valid

NE2 = non-valid

NE3 = valid, egal cu self.cont

Tip tranzactie (T) trebuie sa fie 1 sau 2, deci consideram clasele:

T1 = 1

T2 = 2

T3 = { t | $t \neq 1$ si $t \neq 2$ }

T4 = non numeric

Suma tranzactiei (S) trebuie sa fie pozitiva, iar in cazul platilor trebuie sa fie cel mult egala cu balanta contului:

S1 = { s | $s > 0$ si $s \leq \text{balanta}$ }

S2 = { s | $s > 0$ si $s > \text{balanta}$ }

S3 = { s | $s \leq 0$ }

S4 = non numeric

Vom testa validarea contului pe 8 clase de echivalenta N1 ... N8.

Vom testa procesarea tranzactiilor pe (S1 ... S4) X (T1 ... T3) X (NE1 ... NE3) – 36 clase de echivalenta globale.

In cazul combinatiilor care genereaza tranzactii valide, includem asertiuni pentru a verifica valoarea actualizata a balantei si inregistrarea tranzactiei in istoric. De ex:

```
c.procesare_tranzactie(20.01, 1, "2345678")
self.assertEqual(c.balanta, 28.01)
self.assertIn(Tranzactie(4, 2001, 1, "2345678"), c.tranzactii)
```

rezultate

Cosmic Ray Report

Summary info
Date time: 10/05/2026 21:07:33
Total jobs: 149
Complete: 149 (100.00%)
Surviving mutants: 17 (11.41%)
Job list

Eliminarea a 2 mutanti:

56: Job ID a0b0ad2ece5045a39dfd817e37ddccfe

SURVIVED

worker outcome: WorkerOutcome.NORMAL

test outcome: TestOutcome.SURVIVED

cont.py, start pos: (43, 20), end pos: (43, 22)

operator: core/ReplaceComparisonOperator_Eq_LtE, occurrence: 1, definition_name: procesare_tranzactie

```
--- mutation diff ---
--- acont.py
+++ bcont.py
@@ -40,7 +40,7 @@
     suma_int = int(round(suma * 100))
     if suma_int <= 0:
         raise ValueError("Valoarea tranzactiei trebuie sa fie pozitiva")
-    if not (tip == 1 or tip == 2):
+    if not (tip <= 1 or tip == 2):
         raise ValueError("Tip de tranzactie invalid")
     elif tip == 2 and self._balanta - suma_int < 0:
         raise ValueError(f"Fonduri insuficiente {self.balanta} {suma_int} ")
```

57: Job ID 100a2a72c1dc4715920b4daa3153c6a2

SURVIVED

worker outcome: WorkerOutcome.NORMAL

test outcome: TestOutcome.SURVIVED

cont.py, start pos: (43, 32), end pos: (43, 34)

operator: core/ReplaceComparisonOperator_Eq_LtE, occurrence: 2, definition_name: procesare_tranzactie

```
--- mutation diff ---
--- acont.py
+++ bcont.py
@@ -40,7 +40,7 @@
     suma_int = int(round(suma * 100))
     if suma_int <= 0:
         raise ValueError("Valoarea tranzactiei trebuie sa fie pozitiva")
-    if not (tip == 1 or tip == 2):
+    if not (tip == 1 or tip <= 2):
```

Am adaugat asertiune cu tipul de tranzactie == 0 :

```
with self.assertRaises(Exception): c.procesare_tranzactie(4, 0, "1234557")
```

Rezultatul, mutantii 56 si 57 au fost eliminati:

56 : Job ID 2e90d15aec1d4713ad796f8a561687a2

57 : Job ID 65bdcba7381943abb73a9c0b5e321519

Testul de coverage:

Coverage report: 68%

Files

Functions

Classes

coverage.py v7.13.5, created at 2026-05-10 21:53 +0300

File ▲	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	coverage
cont.py	100%	37	0	0	95%	22	1	98%

Conditia din if este intotdeauna adevarata:

```
id_tranzactie = 1
for t in self.tranzactii:
    if t.id >= id_tranzactie:
        id_tranzactie = t.id + 1
self.tranzactii.append(Tranzactie(id_tranzactie, suma_int, tip, nrcont))
self._balanta += suma_int * (-1 if tip == 2 else 1)
```

Analiza valorilor de frontiera

Pentru N, lungimea numarului de cont valorile de frontiera sunt: 6, 7, 8

N1 = stringuri de 6 cifre

N2 = stringuri de 6 cifre si / sau alte caractere

N3 = stringuri de 7 cifre

N4 = stringuri de 7 cifre si / sau alte caractere

N5 = stringuri de 8 cifre

N6 = stringuri de 8 cifre si / sau alte caractere

N7 = non stringuri de 6 cifre

N8 = non stringuri de 6 cifre si / sau alte caractere

N9 = non stringuri de 7 cifre

N10 = non stringuri de 7 cifre si / sau alte caractere

N11 = non stringuri de 8 cifre

N12 = non stringuri de 8 cifre si / sau alte caractere

Clasele de echivalenta 'externe' pentru valorile valide ale contului rmana aceleasi.

NE1 = valid, mai mic in ordine lexicografica decat self.cont

NE2 = non-valid

NE3 = valid, egal cu self.cont

NE4 = valid, mai mare in ordine lexicografica decat self.cont

Tip tranzactie (T) trebuie sa fie 1 sau 2, deci consideram clasele:

T1 = 0

T2 = 1

T3 = 2

T4 = 3

T5 = non numeric

Suma tranzactiei (S) trebuie sa fie pozitiva, iar in cazul platilor trebuie sa fie cel mult egala cu balanta contului, asadar consideram 2 frontiere: 0 si valoarea balantei contului.

S1 = { s | s > 0 si s = balanta - 0.01 }

S2 = { s | s > 0 si s = balanta }

S3 = { s | s > 0 si s = balanta + 0.01 }

S4 = { s | s = 0.01 }

S5 = { s | s = 0 }

S6 = { s | s = -0.01 }

S7 = non numeric

Pentru functia de validare a numarului de cont vom testa cazurile N1 ... N8

Pentru functia de procesare tranzactii vom testa combinatii de valori pentru (NE1 ... NE4) X (T1 ... T5) X (S1 ... S7)

Rezultate teste

Summary info

Date time: 10/05/2026 23:23:20

Total jobs: 149

Complete: 149 (100.00%)

Surviving mutants: 8 (5.37%)

Mutanii ramasi sunt variante pe care nu stiu sa le elimin prin teste, si care mi se par variante echivalente ale codului, de ex:

28 : Job ID 606b1736de0049c9a12a5319366c2438

SURVIVED
worker outcome: WorkerOutcome.NORMAL
test outcome: TestOutcome.SURVIVED

cont.py, start pos: (53, 34), end pos: (53, 35)

operator: core/ReplaceBinaryOperator_Mul_Div, occurrence: 1, definition_name: procesare_tranzactie

```
--- mutation diff ---  
--- acont.py  
+++ bcont.py  
@@ -50,7 +50,7 @@  
     if t.id >= id_tranzactie:  
         id_tranzactie = t.id + 1  
         self.tranzactii.append(Tranzactie(id_tranzactie, suma_int, tip, nrcont))  
-         self._balanta += suma_int * (-1 if tip == 2 else 1)  
+         self._balanta += suma_int / (-1 if tip == 2 else 1)  
  
# def __str__(self):
```

Coverage report: 63%

Files Functions Classes

coverage.py v7.13.5, created at 2026-05-10 23:21 +0300

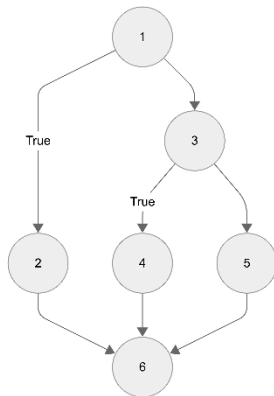
File ▲	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	coverage
cont.py	100%	37	0	0	95%	22	1	98%
test_f.py	43%	122	69	0	92%	12	1	48%
Total	57%	159	69	0	94%	34	2	63%

Testare structurala

Definirea functiei si graful de instructiuni

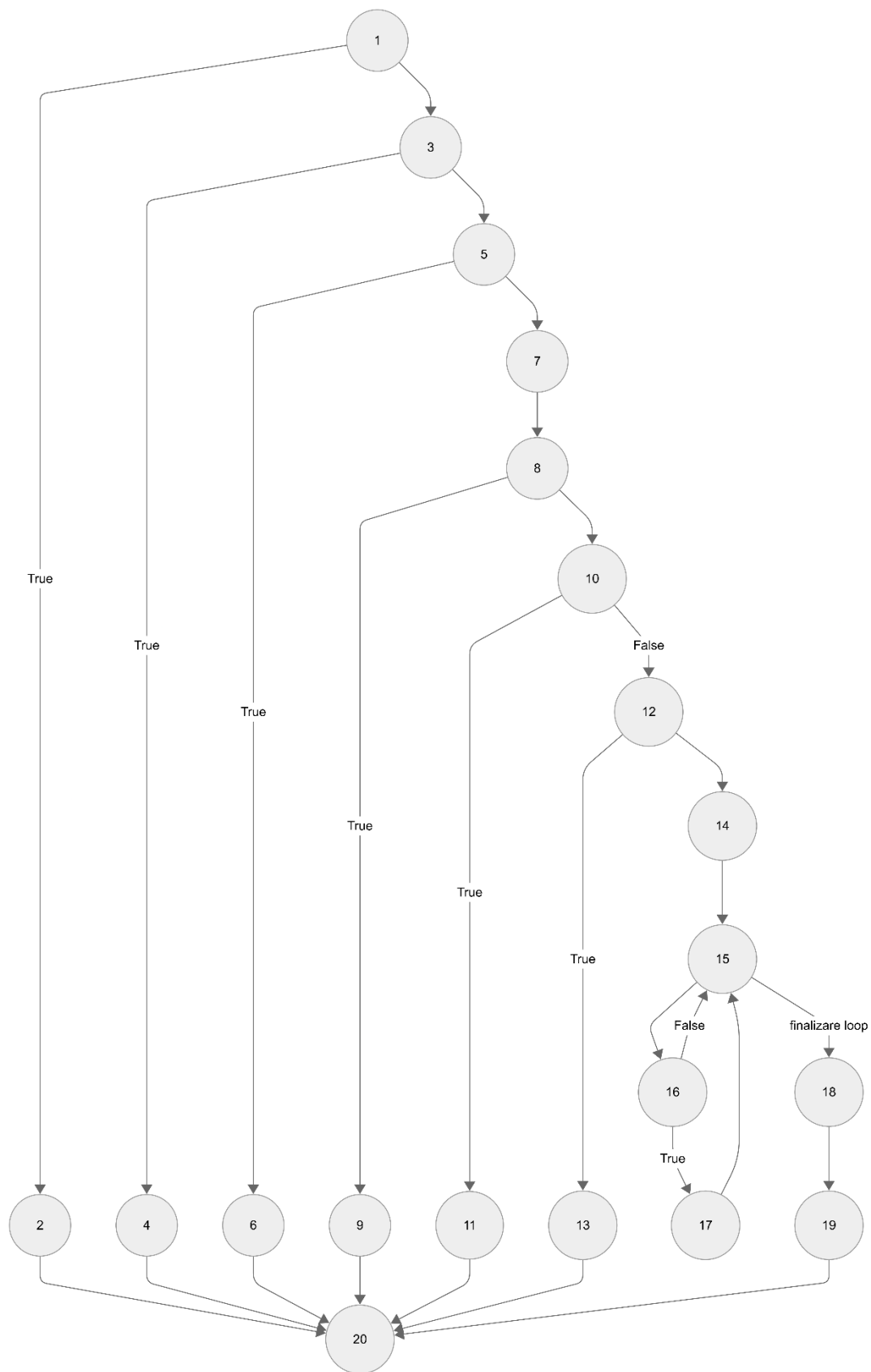
Funcția **cont_valid**

```
def cont_valid(nrcont: str) -> bool:
1     if not isinstance(nrcont, str):
2         return False
3     if len(nrcont) != 7 or not nrcont.isdigit():
4         return False
5     return True
6 (iesirea din functie)
```



Functia `procesare_tranzactie`

```
def procesare_tranzactie(self, suma, tip: int, nrcont: str):
1     if not (isinstance(suma, int) or isinstance(suma, float)):
2         raise TypeError("Suma trebuia sa fie float sau int")
3     if not ContCurent.cont_valid(nrcont):
4         raise ValueError("Numar de cont invalid")
5     if self.nrcont == nrcont:
6         raise ValueError("Conturile sursa si destinatie trebuie sa fie diferite")
7     suma_int = int(round(suma * 100))
8     if suma_int <= 0:
9         raise ValueError("Valoarea tranzactiei trebuie sa fie pozitiva")
10    if not (tip == 1 or tip == 2):
11        raise ValueError("Tip de tranzactie invalid")
12    elif tip == 2 and self._balanta - suma_int < 0:
13        raise ValueError(f"Fonduri insuficiente {self.balanta} {suma_int} ")
14    id_tranzactie = 1
15    for t in self.tranzactii:
16        if t.id >= id_tranzactie:
17            id_tranzactie = t.id + 1
18    self.tranzactii.append(Tranzactie(id_tranzactie, suma_int, tip, nrcont))
19    self._balanta += suma_int * (-1 if tip == 2 else 1)
20    (iesirea din functie)
```



1. Statement coverage

Functia `cont_valid`

Intrare	Rezultat	Instructiuni parcurse
1234567	False	1, 2
"123456"	False	1, 3, 4
"1234567"	True	1, 3, 5

Functia `procesare_tranzactie`

`self.nrcont == "1234567"`

Intrari			Rezultat	Instructiuni parcurse
Suma	Tip	Cont		
"10"	1	"2345678"	Exceptie	1, 2
10	1	"abc"	Exceptie	1, 3, 4
10	1	"1234567"	Exceptie	1, 3, 5, 6
-10	2	"2345678"	Exceptie	1, 3, 5, 7, 8, 9
10	9	"2345678"	Exceptie	1, 3, 5, 7, 8, 10, 11
1000	2	"2345678"	Exceptie (fonduri insuficiente)	1, 3, 5, 7, 8, 10, 12, 13
10	1	"2345678"	Tranzactie procesata. Balanta = 1000 (bani)	1, 3, 5, 7, 8, 10, 12, 14, 15, 18, 19
5	2	"2345678"	Tranzactie procesata. Balanta = 500 (bani)	1, 3, 5, 7, 8, 10, 12, 14, 15, 16, 17, 18, 19 La a doua tranzactie se itereaza in istoric pentru a identifica max(id tranzactie)

Rezultate:

Cosmic Ray Report

Summary info

Date time: 10/05/2026 23:41:18

Total jobs: 149

Complete: 149 (100.00%)

Surviving mutants: 19 (12.75%)

Job list

Coverage report: 69%

[Files](#)[Functions](#)[Classes](#)

coverage.py v7.13.5, created at 2026-05-10 23:42 +0300

File ▲	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	coverage
cont.py	100%	37	0	0	95%	22	1	98%
test_s.py	40%	57	34	0	50%	2	1	41%
Total	64%	94	34	0	92%	24	2	69%

coverage.py v7.13.5, created at 2026-05-10 23:42 +0300

2. Decision coverage

Funcția **cont_valid**

	Decizii
1	not isinstance(nrcont, str)
2	len(nrcont) != 7 or not nrcont.isdigit()

Intrare	Rezultat	Decizii acoperite
1234567	False	1 - false
"123456"	False	1 - true, 2 - false
"1234567"	True	1 - true, 2 - true

NOTA: aceleasi intrari pentru **statement coverage** asigura si **decision coverage**

Funcția **procesare_tranzactie**

	Decizii
1	not (isinstance(suma, int) or isinstance(suma, float))
2	not ContCurent.cont_valid(nrcont)
3	if self.nrcont == nrcont
4	suma_int <= 0
5	not (tip == 1 or tip == 2)
6	tip == 2 and self._balanta - suma_int < 0
7	for t in self.tranzactii
8	if t.id >= id_tranzactie <i>in mod normal istoricul tranzactiilor este in ordine crescatoare dupa t.id, deci urmatorul t.id va fi cel putin egal cu id_tranzactie – conditia va fi in general adevarata</i>

self.nrcont == "1234567"

Intrari			Rezultat	Decizii acoperite
Suma	Tip	Cont		
"10"	1	"2345678"	Exceptie	1 - false
10	1	"abc"	Exceptie	1 - true, 2 - false
10	1	"1234567"	Exceptie	1,2 - true, 3 - false
-10	2	"2345678"	Exceptie	1...3 - true, 4- false
10	9	"2345678"	Exceptie	1...4 - true, 5- false
1000	2	"2345678"	Exceptie (fonduri insuficiente)	1...6 - true
10	1	"2345678"	Tranzactie procesata. Balanta = 1000 (bani)	1...5 - true, 6 - false, 7 - false
5	2	"2345678"	Tranzactie procesata. Balanta = 500 (bani)	1...5 - true, 6 - false, 7 - true, 8 - true

NOTA: aceleasi intrari pentru **statement coverage** asigura si **decision coverage**

3. Condition coverage

Funcția **cont_valid**

	Decizii		Conditii individuale
1	not isinstance(nrcont, str)	1	not isinstance(nrcont, str)
2	len(nrcont) != 7 or not nrcont.isdigit()	2	len(nrcont) != 7
		3	not nrcont.isdigit()

Intrare	Rezultat	Conditii acoperite
1234567	False	1 - false
"123456"	False	1 - true, 2 - false, 2 - true
"123456a"	False	1 - true, 2 - true, 2 - false
"1234567"	True	1 - true, 2 - true, 2 - true

Funcția **procesare_tranzactie**

	Decizii		Conditii individuale
1	not (isinstance(suma, int) or isinstance(suma, float))	1	isinstance(suma, int)
		2	isinstance(suma, float))
2	not ContCurent.cont_valid(nrcont)	3	not ContCurent.cont_valid(nrcont)
3	if self.nrcont == nrcont	4	if self.nrcont == nrcont
4	suma_int <= 0	5	suma_int <= 0
5	not (tip == 1 or tip == 2)	6	tip == 1
		7	tip == 2
6	tip == 2 and self._balanta - suma_int < 0	8	tip == 2
		9	self._balanta - suma_int < 0
7	for t in self.tranzactii	10	for t in self.tranzactii
8	if t.id >= id_tranzactie <i>in mod normal istoricul tranzactiilor este in ordine crescatoare dupa t.id, deci urmatorul t.id va fi cel putin egal cu id_tranzactie – conditia va fi in general adevarata</i>	11	if t.id >= id_tranzactie

Observatie: conditiile individuale 1, 2 si 6, 7 pot fi acoperite fara a asigura branch coverage adecvat, daca in teste sunt alternativ false si lipseste un test in care ambele sunt false.

self.nrcont == "1234567"

Intrari			Rezultat	Conditii individuale acoperite	
Suma	Tip	Cont		Adevarate	False
10	1	"123"	Exceptie	1	2, 3
10.0	1	"123"	Exceptie	2	1, 3
10	1	"1234567"	Exceptie	3	4
-10	1	"2345678"	Exceptie	4, 5	
10	9	"2345678"	Exceptie		5, 6, 7

1000	2	"2345678"	Exceptie (fonduri insuficiente)	7, 8, 9	
10	1	"2345678"	Tranzactie procesata. Balanta = 1000 (bani)	6	8, 9, 10
5	2	"2345678"	Tranzactie procesata. Balanta = 500 (bani)	10, 11	

Rezultate

Cosmic Ray Report

Summary info

Date time: 10/05/2026 23:45:16

Total jobs: 149

Complete: 149 (100.00%)

Surviving mutants: 19 (12.75%)

Coverage report: 69%

Files

Functions

Classes

coverage.py v7.13.5, created at 2026-05-10 23:47 +0300

File ▲	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	coverage
cont.py	97%	37	1	0	91%	22	2	95%
test_s.py	42%	57	33	0	50%	2	1	42%
Total	64%	94	34	0	88%	24	3	69%

coverage.py v7.13.5, created at 2026-05-10 23:47 +0300

4. Testarea circuitelor independente

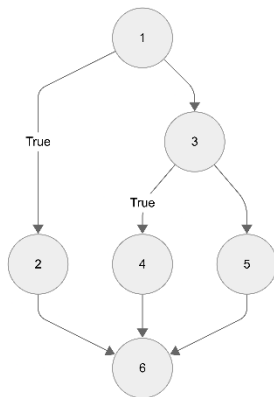
Funcția **cont_valid**

$e = 7$

$n = 6$

$$V(G) = 7 - 6 + 2 = 3$$

```
def cont_valid(nrcont: str) -> bool:
1     if not isinstance(nrcont, str):
2         return False
3     if len(nrcont) != 7 or not nrcont.isdigit():
4         return False
5     return True
6     (iesirea din functie)
```



Circuite independente:

1) 1, 2, 6

2) 1, 3, 4, 6

3) 1, 3, 5, 6

Intrare	Rezultat	Circuit
1234567	False	1) 1, 2, 6
"123456"	False	2) 1, 3, 4, 6
"1234567"	True	3) 1, 3, 5, 6

Funcția **procesare_tranzactie**

$e = 27$

$n = 20$

$$V(G) = e - n + 2 = 27 - 20 + 2 = 9$$

```
def procesare_tranzactie(self, suma, tip: int, nrcont: str):
1     if not (isinstance(suma, int) or isinstance(suma, float)):
2         raise TypeError("Suma trebuia sa fie float sau int")
3     if not ContCurent.cont_valid(nrcont):
4         raise ValueError("Numar de cont invalid")
```

```

5     if self.nrcont == nrcont:
6         raise ValueError("Conturile sursa si destinatie trebuie sa fie diferite")
7     suma_int = int(round(suma * 100))
8     if suma_int <= 0:
9         raise ValueError("Valoarea tranzactiei trebuie sa fie pozitiva")
10    if not (tip == 1 or tip == 2):
11        raise ValueError("Tip de tranzactie invalid")
12    elif tip == 2 and self._balanta - suma_int < 0:
13        raise ValueError(f"Fonduri insuficiente {self.balanta} {suma_int} ")
14    id_tranzactie = 1
15    for t in self.tranzactii:
16        if t.id >= id_tranzactie:
17            id_tranzactie = t.id + 1
18    self.tranzactii.append(Tranzactie(id_tranzactie, suma_int, tip, nrcont))
19    self._balanta += suma_int * (-1 if tip == 2 else 1)
20    (iesirea din functie)

```

Circuite independente:

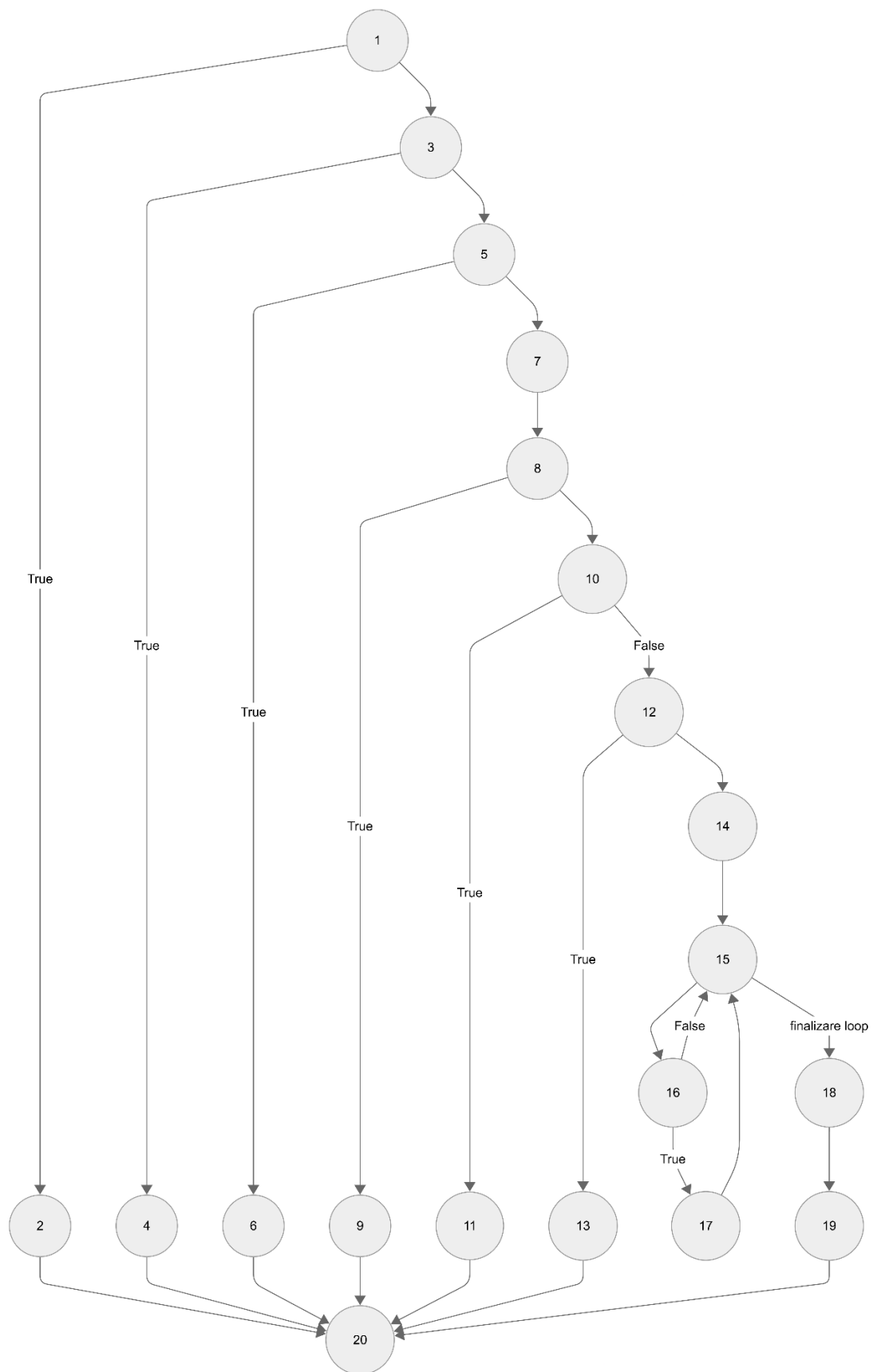
- 1) 1, 2, 20
- 2) 1, 3, 4, 20
- 3) 1, 3, 5, 6, 20
- 4) 1, 3, 5, 7, 8, 9, 20
- 5) 1, 3, 5, 7, 8, 9, 10, 11, 20
- 6) 1, 3, 5, 7, 8, 9, 10, 12, 13, 20
- 7) 1, 3, 5, 7, 8, 9, 10, 12, 14, 15, 18, 19, 20
- 8) 1, 3, 5, 7, 8, 9, 10, 12, 14, 15, 16, 15, 18, 19, 20
- 9) 1, 3, 5, 7, 8, 9, 10, 12, 14, 15, 16, 17, 15, 18, 19, 20

*NOTA: in mod normal istoricul tranzactiilor este in ordine crescatoare in functie de t.id, deci urmatorul t.id va fi cel putin egal cu id_tranzactie – conditia va fi in general adevarata. In aceste conditii ramura 16-15 nu poate fi parcursa, deci **circuitul 8 nu poate fi acoperit***

self.nrcont == "1234567"

Intrari			Rezultat	Circuit
Suma	Tip	Cont		
"10"	1	"2345678"	Exceptie	1) 1, 2, 20
10	1	"abc"	Exceptie	2) 1, 3, 4, 20
10	1	"1234567"	Exceptie	3) 1, 3, 5, 6, 20
-10	2	"2345678"	Exceptie	4) 1, 3, 5, 7, 8, 9, 20
10	9	"2345678"	Exceptie	5) 1, 3, 5, 7, 8, 9, 10, 11, 20
1000	2	"2345678"	Exceptie (fonduri insuficiente)	6) 1, 3, 5, 7, 8, 9, 10, 12, 13, 20
10	1	"2345678"	Tranzactie procesata. Balanta = 1000 (bani)	7) 1, 3, 5, 7, 8, 9, 10, 12, 14, 15, 18, 19, 20

5	2	"2345678"	Tranzactie procesata. Balanta = 500 (bani)	9) 1, 3, 5, 7, 8, 9, 10, 12, 14, 15, 16, 17, 15, 18, 19, 20
---	---	-----------	---	--



Rezultate teste:

Cosmic Ray Report

Summary info

Date time: 10/05/2026 23:50:11

Total jobs: 149

Complete: 149 (100.00%)

Surviving mutants: 19 (12.75%)

Job list

Coverage report: 69%

Files Functions Classes

coverage.py v7.13.5, created at 2026-05-10 23:51 +0300

File ▲	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	coverage
cont.py	100%	37	0	0	95%	22	1	98%
test_s.py	40%	57	34	0	50%	2	1	41%
Total	64%	94	34	0	92%	24	2	69%

coverage.py v7.13.5, created at 2026-05-10 23:51 +0300